



EcoRide

Documentation technique

Version 1.0 – Novembre 2025

Table des matières

- 1 . Choix technologiques
2. Structure du projet
3. Configuration de l'environnement
4. Modèle conceptuel de données
5. Diagrammes d'utilisation et de séquence
6. Étapes de déploiement
7. Identifiants de test

ECORIDE

1. Choix technologiques l'application

Le développement de l'application EcoRide a été guidé par des choix technologiques visant un équilibre entre la robustesse, la rapidité de développement et la conformité avec les exigences du cahier des charges.

- **Langage Serveur (Backend) : PHP natif**

- **Choix** : L'application a été développée en PHP natif (version 8.1+) en suivant une architecture de type "Contrôleur Frontal" inspirée du modèle MVC.
- **Justification** : Ce choix a été fait pour démontrer une maîtrise des concepts fondamentaux du développement web backend (routage, gestion des requêtes HTTP, séparation de la logique et de l'affichage) sans dépendre d'un framework majeur comme Symfony ou Laravel. L'utilisation de **PDO** (PHP Data Objects) garantit un accès sécurisé et standardisé à la base de données.

- **Interface Utilisateur (Frontend) : HTML, CSS, Bootstrap & JavaScript**

- **Choix** : L'interface a été construite avec une stack classique et éprouvée.

- **Justification** :

Bootstrap 5 a été choisi pour son système de grille puissant et ses composants prêts à l'emploi (comme la barre de navigation, les cartes de trajet et les modales). Cela a permis de développer rapidement une interface responsive et moderne.

Sass (SCSS) a été utilisé pour rendre le code CSS plus maintenable. Grâce aux **variables**, la charte graphique est centralisée ; grâce à **l'imbrication** (nesting), le code est plus lisible ; et grâce aux **partials** (_header.scss, _covoiturage.scss), les styles sont organisés en fichiers logiques.

Le **JavaScript natif** est utilisé pour les interactions dynamiques (appels API via fetch, gestion des modales), gardant l'application légère sans la complexité d'un framework front-end lourd.

- **Bases de Données : Approche Hybride (MySQL & MongoDB)**

- **Choix** : Conformément à une exigence clé de l'énoncé, une architecture de données hybride a été mise en place.

- **Justification** :

MySQL a été choisi pour la partie relationnelle en raison de sa grande fiabilité, de sa robustesse et de ses garanties transactionnelles (ACID), ce qui est idéal pour les données structurées et critiques (utilisateurs, trajets, réservations).

MongoDB a été utilisé pour la partie non-relationnelle afin de gérer les préférences des utilisateurs. Sa flexibilité et sa structure "sans schéma" sont parfaitement adaptées pour stocker une liste de préférences qui peut inclure des valeurs

personnalisées ajoutées par l'utilisateur, sans nécessiter de modification de la structure de la base.

- **Gestion des Dépendances : Composer & NPM**

- **Choix** : Composer pour le PHP et NPM pour le JavaScript sont les standards de l'industrie.
- **Justification** : Leur utilisation permet de gérer proprement les librairies externes (comme PHPMailer ou Bootstrap), d'assurer des installations reproductibles grâce aux fichiers de "lock", et de faciliter la maintenance du projet.



2. Structure du projet

L'application EcoRide est structurée selon le patron d'architecture "**Front Controller**" (Contrôleur Frontal), avec une organisation des dossiers inspirée du modèle **MVC (Modèle-Vue-Contrôleur)**. Toutes les requêtes HTTP sont interceptées et redirigées par le serveur web vers un point d'entrée unique, le fichier `index.php`, qui agit comme un routeur principal.

L'arborescence du projet est organisée comme suit :

- **/assets/**
Contient tous les fichiers statiques (ressources front-end) comme les feuilles de style CSS compilées et le dossier images.
- **/Node-modules/**
Contient tous les fichiers BOOTSTRAP (ignoré par Git).
- **/src/**
C'est le répertoire principal du code source de l'application en PHP.
 - **/src/Controllers/** : Contient les classes des **Contrôleurs**. Chaque contrôleur regroupe la logique métier pour une section de l'application (ex: UserController, TrajetController). Ils reçoivent les requêtes du routeur, interagissent avec la base de données, et préparent les données pour les vues.
 - **/src/Core/** : Contient les classes fondamentales de l'application, notamment Database.php, qui utilise le design pattern Singleton pour gérer une connexion unique et efficace aux bases de données MySQL et MongoDB.
 - **/src/Models/** : Contient les classes des Modèles (ex: UserModel, TrajetModel). C'est la seule couche de l'application autorisée à communiquer avec la base de données
- **/vendor/**
Ce dossier est géré automatiquement par **Composer**. Il contient toutes les librairies PHP externes téléchargées et n'est pas inclus dans le dépôt Git.
- **/views/**
 - Ce dossier contient toutes les **Vues** : Ce sont des fichiers .php dont le rôle est d'afficher le code HTML et les données préparées par les contrôleurs pour construire l'interface utilisateur.
- **header_template.php** : Fichier de template contenant l'en-tête HTML commun à toutes les pages, y compris la barre de navigation principale et le chargement des feuilles de style.
- **footer_template.php** : Fichier de template pour le pied de page commun, qui contient également le code HTML de la modale de réservation et les appels aux scripts JavaScript globaux.
- **index.php** : Le **Contrôleur Frontal**. C'est le cœur de l'application qui reçoit toutes les requêtes, analyse l'URL demandée, et appelle le bon contrôleur pour exécuter la logique appropriée.
- **.htaccess** : Fichier de configuration pour le serveur web Apache. Son rôle est crucial : il redirige toutes les URL non existantes physiquement (comme /profile ou /login) vers le fichier `index.php`.
- **composer.json** : Fichier de manifeste qui déclare toutes les dépendances PHP du projet (PHPMailer, Dotenv, etc.), gérées par Composer.
- **package.json** : Fichier de manifeste pour les dépendances front-end (Bootstrap, Bootstrap Icons), gérées par npm.
- **.env** : Fichier de configuration local (ignoré par Git) contenant les variables d'environnement sensibles.
- **Readme.md** : Le document principal du projet, présentant l'application et fournissant les instructions détaillées pour l'installation en environnement de développement local.

Cette structure permet une séparation claire des préoccupations (logique, affichage, configuration), ce qui rend le projet plus facile à maintenir et à faire évoluer.

3. Configuration de l'environnement

Pour exécuter le projet en local, suivez ces étapes :

1. **Cloner le dépôt :**

```
Bash
git clone https://github.com/Fani1987/EcoRide.git
cd EcoRide
```

2. **Installer les dépendances :**

```
Bash
composer install
npm install
```

3. **Configurer les variables d'environnement :**

- Créez un fichier `.env` à la racine du projet en copiant le modèle `.env.example` (si vous en avez un) ou en le créant manuellement. Remplissez-le avec vos informations locales :

```
# Base de données MySQL
○ DB_HOST=localhost
○ DB_NAME=ecoride
○ DB_USER=root
○ DB_PASS=
○ DB_CHARSET=utf8mb4

# Base de données MongoDB
○ MONGO_DB_HOST=localhost
○ MONGO_DB_PORT=27017
○ MONGO_DB_NAME=ecoride
○ MONGO_DB_USERNAME=
○ MONGO_DB_PASSWORD=

# Service d'e-mails (exemple avec Mailtrap pour le test)
○ MAIL_HOST=sandbox.smtp.mailtrap.io
○ MAIL_USERNAME=xxxxxxxxxxxxxx
○ MAIL_PASSWORD=xxxxxxxxxxxxxx
○ MAIL_PORT=2525
```

4. **Importer la base de données :**

- Créez une base de données nommée `ecoride` dans votre gestionnaire de base de données (phpMyAdmin).
- Importez le fichier `ecoride.sql` (fourni à la racine du projet) dans cette base de données.

5. **Configurer le serveur web (Très important) :**

- Configurez un **Hôte Virtuel** (Virtual Host) dans votre environnement WAMP ou XAMPP qui pointe vers le dossier racine de votre projet.
- Accédez ensuite au site via l'URL que vous avez définie (ex: `http://ecoride.local`). Cela est nécessaire pour que le routage des URL (`/profile`, `/login`, etc.) fonctionne correctement grâce au fichier `.htaccess`.

4. Modèle conceptuel de données

L'architecture des données d'EcoRide repose sur une approche hybride, utilisant à la fois une base de données relationnelle (MySQL) et une base de données non-relationnelle (NoSQL - MongoDB). Cette conception permet de tirer parti des forces de chaque technologie : la structure et la fiabilité de SQL pour les données critiques, et la flexibilité de NoSQL pour les données semi-structurées.

○ Base de Données Relationnelle (MySQL)

Le cœur de l'application est géré par une base de données MySQL. Elle garantit l'intégrité et la cohérence des données transactionnelles grâce à un schéma strict et à l'utilisation de clés étrangères.

Les tables principales sont :

- **utilisateurs** : C'est la table centrale qui stocke les informations de tous les comptes, qu'il s'agisse de passagers, de chauffeurs, d'employés ou d'administrateurs. Elle contient les identifiants de connexion, le rôle, le solde de crédits et la note moyenne pour les chauffeurs.
- **covoiturages** : Contient toutes les offres de trajet publiées par les chauffeurs. La colonne statut (planifié, en_cours, terminé, annulé) est essentielle pour gérer le cycle de vie de chaque trajet.
- **vehicules** : Stocke les informations sur les véhicules des chauffeurs. Un chauffeur peut posséder plusieurs véhicules.
- **reservations** : Table de liaison qui matérialise la participation d'un passager à un covoiturage. Sa propre colonne statut (en_attente, confirmée, validée, annulée, etc.) est cruciale pour gérer le processus de réservation, de la demande initiale à la validation finale.
- **avis** : Contient les notes et commentaires laissés par les passagers. Son champ statut permet de mettre en place le flux de modération par les employés.
- **incidents et notifications** : Tables de support qui permettent respectivement de gérer les litiges et d'envoyer des informations aux utilisateurs sur leur profil. Des **clés étrangères** et des contraintes d'intégrité (comme ON DELETE CASCADE) sont utilisées pour garantir que les relations entre ces tables restent toujours cohérentes.

○ Base de Données Non-Relationnelle (MongoDB)

Pour répondre au besoin de flexibilité des préférences de voyage des utilisateurs (US 8), une base de données NoSQL a été mise en place avec MongoDB.

- **Objectif** : Gérer les préférences des utilisateurs, qui peuvent être à la fois des options standards (Fumeur, Animal, Musique) et des valeurs personnalisées ajoutées par l'utilisateur ("Discussions bienvenues", "Petit bagage", etc.).
- **Structure** : Une collection unique nommée preferences est utilisée. Chaque document de cette collection correspond à un utilisateur et contient deux champs principaux :
 - mysql_user_id : L'ID de l'utilisateur provenant de la table SQL utilisateurs, qui sert de lien entre les deux bases de données.
 - preferences : Un tableau de chaînes de caractères contenant la liste des préférences de l'utilisateur.
- **Avantage** : Cette approche "sans schéma" permet d'ajouter n'importe quelle préférence personnalisée sans jamais avoir à modifier la structure de la base de données, offrant une flexibilité maximale.

En conclusion, cette architecture de données hybride permet à l'application EcoRide d'être à la fois robuste et structurée pour ses données essentielles, tout en étant souple et évolutive pour les données plus dynamiques comme les préférences des utilisateurs.

Démonstration d'une Transaction Métier (Validation de Trajet)

Pour illustrer l'architecture MVC et la gestion transactionnelle, le cas d'usage le plus pertinent est la **validation d'un trajet** par un passager (géré par `AvisController::validateTrajet`).

Cette action n'est pas une simple requête SQL, mais une **transaction métier** qui orchestre plusieurs modèles pour garantir l'intégrité des données.

Objectif Métier : Lorsqu'un passager valide un trajet, le système doit impérativement :

1. Mettre à jour le statut de sa réservation.
2. **Payer le chauffeur** en transférant les crédits.
3. Enregistrer l'avis du passager pour modération.

Sécurité (Transaction PDO) : Ces trois actions doivent réussir ou échouer *ensemble*. Il est impensable de payer le chauffeur si l'enregistrement de l'avis échoue. Pour garantir cela, l'ensemble du bloc est entouré d'une transaction PDO :

- `$pdo->beginTransaction()` ; est appelé au début.
- Si tout réussit, `$pdo->commit()` ; valide les changements.
- Si une seule erreur survient, un `catch (Exception $e)` exécute un `$pdo->rollBack()` ; pour annuler toutes les opérations, assurant qu'aucun crédit n'est transféré à tort.

Orchestration (Le Contrôleur) : Le `AvisController` ne contient aucun SQL. Il agit en chef d'orchestre et appelle les méthodes de différents modèles pour accomplir la tâche :

Extrait simplifié de `AvisController::validateTrajet()` :

```
<?php
public static function validateTrajet(PDO $pdo, int $reservation_id, array
$avisData)
{
    // (Ici, on récupère $passagerId depuis $_SESSION)

    try {
        // 1. Démarrer la transaction
        $pdo->beginTransaction();

        // 2. Instancier les modèles
        $reservationModel = new ReservationModel($pdo);
        $avisModel = new AvisModel($pdo);
        $userModel = new UserModel($pdo);

        // 3. VÉRIFIER (Appel Modèle 1)
        // Le contrôleur vérifie si l'utilisateur a le droit
        $reservation = $reservationModel->findForValidation($reservation_id,
$passagerId);
        if (!$reservation) {
```

```

        throw new \Exception('Impossible de valider ce trajet.');
```

```

    }

    // 4. EXÉCUTER (Appels Modèles 2, 3 et 4)

    // a. Mettre à jour la réservation
    $reservationModel->updateStatus($reservation_id, 'validée');

    // b. Insérer l'avis
    $avisModel->create($passagerId, $reservation['covoiturage_id'],
(int)$avisData['note'], $avisData['commentaire']);

    // c. Payer le chauffeur
    $userModel->creditCredits($reservation['chauffeur_id'],
$reservation['prix']);

    // 5. Valider la transaction
    $pdo->commit();
    echo json_encode(['success' => true, 'message' => 'Trajet validé !']);

} catch (\Exception $e) {
    // 6. Annuler en cas d'erreur
    $pdo->rollBack();
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);
}
}

```

Cette approche démontre la puissance de l'architecture MVC : le Contrôleur gère la logique métier complexe (la transaction) tandis que les Modèles gèrent l'exécution SQL simple (les `UPDATE`, `INSERT`).



5. Diagrammes d'utilisation et de séquence (annexes)

○ Diagramme de Cas d'Utilisation

Ce diagramme offre une vue d'ensemble des fonctionnalités de la plateforme en identifiant les différents types d'utilisateurs, appelés "**acteurs**", et les actions, ou "**cas d'utilisation**", qu'ils sont autorisés à effectuer. Il permet de comprendre rapidement le périmètre fonctionnel de l'application et les droits associés à chaque rôle.

Les acteurs identifiés sont :

- **Visiteur** : Toute personne non authentifiée. Il peut rechercher et consulter les trajets et les profils publics des chauffeurs, ainsi que s'inscrire ou se connecter.
- **Passager / Chauffeur** : Un utilisateur connecté. Selon son profil, il hérite des droits du visiteur et peut en plus :
 - **En tant que Passager** : Réserver des trajets, annuler ses réservations, acheter des crédits, et surtout valider un trajet terminé pour laisser un avis et déclencher le paiement du chauffeur.
 - **En tant que Chauffeur** : Gérer ses véhicules, proposer des trajets, gérer les demandes de réservation (confirmer/refuser), et piloter le cycle de vie de ses trajets (démarrer, terminer, annuler).
- **Employé** : Un utilisateur avec des droits de modération, chargé de valider ou refuser les avis soumis par les passagers et de traiter les incidents signalés.
- **Administrateur** : Le rôle avec le plus haut niveau de privilèges, qui peut créer les comptes des employés, gérer tous les comptes utilisateurs et visualiser les statistiques de la plateforme.

○ Diagramme de Séquence

Le diagramme de séquence a pour but d'illustrer les interactions entre les différents composants du système (l'utilisateur, le navigateur, les contrôleurs, la base de données) au cours d'un scénario précis. Le scénario modélisé est celui de la **validation d'un trajet par le passager après sa finalisation par le chauffeur**. Ce flux est l'un des plus importants de l'application car il met en jeu plusieurs acteurs et déclenche la transaction financière (le transfert de crédits).

Le déroulement de la séquence est le suivant :

1. Le **Chauffeur**, via son interface, signale la fin du trajet.
2. Le Mapsur envoie un appel API à la route `/api/endTrajet`, qui est traitée par le `TrajetController`.
3. Le `TrajetController` met à jour le statut du covoiturage à "terminé" dans la `BaseDeDonnees`.
4. Il récupère ensuite la liste des passagers et, pour chacun, crée une Notification interne et simule l'envoi d'un e-mail.
5. Plus tard, le **Passager** se connecte à son profil. L'interface demande au `UserController` les informations de ses trajets réservés.
6. Le contrôleur récupère les données et la vue affiche le formulaire de validation pour le trajet qui est maintenant "terminé".
7. Le **Passager** soumet son avis et sa note.
8. Le Mapsur envoie un appel API à la route `/api/validateTrajet`, traitée par l'`AvisController`.
9. L'`AvisController` exécute une transaction sécurisée sur la `BaseDeDonnees` pour :
 - Mettre à jour le statut de la réservation à "validée".
 - Enregistrer le nouvel avis avec un statut "en_attente" de modération.

- Transférer les crédits du trajet au compte du chauffeur.

10. Le système retourne une confirmation de succès, et l'interface du passager est mise à jour.



6. Étapes de déploiement

Cette section décrit la démarche et les étapes suivies pour déployer l'application EcoRide sur un serveur de production, la rendant ainsi accessible en ligne.

○ Choix de la Plateforme d'Hébergement

Pour ce projet, la plateforme **Railway.app** a été choisie. Cette solution de type PaaS (Platform as a Service) a été sélectionnée pour plusieurs raisons :

- **Simplicité** : Elle permet de déployer une application directement depuis un dépôt GitHub avec une configuration minimale.
- **Offre Gratuite Complète** : Son plan gratuit est suffisant pour héberger les trois composants de notre stack : l'application PHP, la base de données MySQL, et la base de données MongoDB.
- **Gestion Centralisée** : Tous les services sont gérés au sein d'un seul et même projet, ce qui simplifie la configuration du réseau et des variables d'environnement.

○ Préparation du Projet pour le Déploiement

Avant de lancer le déploiement, plusieurs étapes de préparation du code ont été nécessaires pour assurer la compatibilité avec un environnement de production :

- **Configuration des dépendances (composer.json)** : Le fichier a été simplifié pour ne déclarer que les librairies PHP essentielles. Les dépendances d'extensions PHP (ext-*) ont été retirées pour laisser la plateforme de déploiement gérer l'installation de l'environnement PHP de base.
- **Gestion du Fichier composer.lock** : Le fichier composer.lock a été inclus dans le dépôt Git. C'est une étape cruciale qui garantit que le serveur de déploiement installe exactement les mêmes versions des dépendances que celles utilisées en développement, assurant ainsi la stabilité.
- **Préparation du fichier SQL (ecoride.sql)** : Le fichier d'exportation de la base de données a été modifié pour garantir une importation fiable :
 - Les TRIGGER (déclencheurs) ont été retirés pour éviter des erreurs de syntaxe avec certains clients SQL.
 - Les commandes SET FOREIGN_KEY_CHECKS=0; et SET FOREIGN_KEY_CHECKS=1; ont été ajoutées au début et à la fin du fichier pour contourner les problèmes d'ordre lors de la création des tables.
- **Gestion des Variables d'Environnement (.env)** : Le fichier index.php a été modifié pour que le chargement du fichier .env soit conditionnel. Ainsi, en local, les variables du fichier sont utilisées, tandis qu'en production, l'application utilise les variables fournies par la plateforme.

○ Processus de Déploiement sur Railway.app

Le déploiement s'est déroulé en suivant ces étapes chronologiques :

- **Création du Projet et des Services** : Un projet vide a été créé sur Railway. Ensuite, les trois services nécessaires ont été ajoutés : une base de données MySQL, une base de données MongoDB, et l'application PHP déployée depuis le dépôt GitHub Fani1987/EcoRide.
- **Importation des Données** : Une connexion a été établie avec la base de données MySQL en ligne via un client SQL externe (HeidiSQL). Le fichier ecoride.sql a été exécuté pour créer la structure des tables et insérer les données de test. Cette étape a nécessité la configuration d'une connexion sécurisée (SSL).
- **Configuration des Variables d'Environnement** : C'est l'étape la plus critique. Dans l'onglet "Variables" du service applicatif sur Railway, toutes les variables d'environnement ont été configurées (DB_HOST,

DB_USER, DB_PASS, DB_PORT, MONGO_..., MAIL_...) en utilisant les identifiants de connexion fournis par les services de base de données Railway.

- **Configuration de la Commande de Démarrage** : Dans les réglages du service applicatif, la "Start Command" a été définie sur `php -S 0.0.0.0:$PORT -t ..`. Cette commande lance le serveur web intégré de PHP et lui indique d'utiliser le port fourni par Railway, assurant que l'application est accessible depuis l'extérieur.

- **Vérification Finale**

Après un déploiement réussi, un domaine public (ex: <https://ecoride-production-d9b8.up.railway.app>) a été généré via les réglages du service. L'application a ensuite été testée de fond en comble sur cette adresse en ligne pour valider le bon fonctionnement de toutes les fonctionnalités : inscription, connexion, recherche, réservation et les espaces spécifiques aux différents rôles.

Méthodologie de Gestion de Version (Git)

Le simple fait d'utiliser Git n'est pas suffisant ; il est crucial d'adopter un flux de travail (workflow) professionnel pour garantir la stabilité du projet.

Conformément aux bonnes pratiques, le projet n'a pas été développé sur une seule branche main, ce qui est risqué. J'ai appliqué un flux de travail basé sur les branches develop et main :

1. **Branche main** : Elle est protégée. Elle ne contient que le code de production stable. Le déploiement sur Railway était initialement lié à cette branche.
2. **Branche develop** : C'est la branche de travail principale. La plupart des 29 commits du projet s'y trouvent.
3. **Branches de feature / hotfix** : Pour toute nouvelle fonctionnalité ou correction (comme la mise en place du JavaScript de réservation), le travail est fait sur une branche dédiée (ex: hotfix/modal-reservations).

Processus de mise à jour :

1. Une branche hotfix est créée depuis main (ou develop).
2. Le travail est effectué et "commite" sur cette branche.
3. La branche est fusionnée (merge) dans develop.
4. La branche develop est "pushée" sur GitHub.
5. Railway, qui est configuré pour surveiller la branche develop, détecte le push et déploie automatiquement la nouvelle version sur le site en ligne